

Problem 1: Lesion Classification Random Forest

1.1 Feature Construction Description Our feature representation combines tabular features from the data, along with two engineered features created from the image data. We used the scikitlearn OneHotEncoder function to create binary features for categories that contained strings as inputs in the dataset, such as 'region'. For values that were missing/unfilled, we maintained the original implementation and replaced missing values as '0' or 'false'.

Our engineered image features:

1. **Color Variation:** We calculated the standard deviation of each color channel in the images to get a sense of the color variation. This created three separate features we used in the model, the standard deviation of the red channel, the green channel, and the blue channel. The idea behind this was that harmful lesions would have more color variation.
2. **Region Contrast:** Using Otsu thresholding provided by the OpenCV library, we found the optimal threshold that separates the image into two sections, ideally the area of the lesion and the area surrounding the lesion. We then calculated the average difference in intensity between the pixels in the two regions as the numerical feature to use in our model. The idea behind this was that if the difference in intensity was greater, there was a higher chance that a skin lesion was actually present.

In total, the model takes in the 4 engineered features, as well as 32 (after performing one hot encoding) tabular features from the dataset. The only features that we did not include from the original tabular data was the information regarding the patients parents country of origin.

We did not use any scaling or normalization on the input numerical features, as the random forest model would not benefit from it due to thresholds being made from relative relationships between features and not their magnitude.

1.2 Cross Validation Design Description Because our dataset contains multiple lesions from the same patients, a standard cross validation does not take into consideration where visually similar lesions from the same patient appears in both the training and validation folds. To compensate and prevent the model from memorizing the individual patients, we implement a 5-fold **StratifiedGroupKFold** (implemented using the scikitlearn library) strategy, using the `patient_id` as the grouping variable. In doing so, we ensure that all images belonging to the same patient were kept in the same fold, and each class is balanced across all splits:

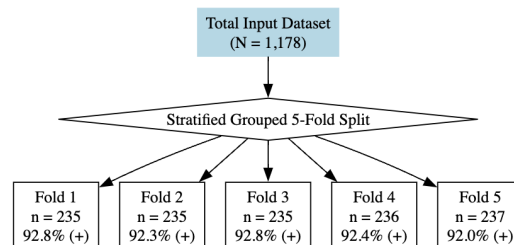


Figure 1: Stratified Group K Fold Split Visualization

`roc_auc` was the performance metric that our search will optimize on the heldout data. After cross-validation identifies the best-performing hyperparameter configuration, `GridSearchCV` automatically re-fits the entire pipeline on the full training dataset using those selected hyperparameters. This refit model becomes our final trained classifier. We then evaluate this single final model once on the test set to obtain an unbiased estimate of performance.

1.3 Hyperparameter Selection for Random Forest Classifier We optimize our model over four hyperparameters: `max_features`, `max_depth`, `min_samples_leaf`, and `n_estimators`. This is done with `GridSearchCV` (from `scikitlearn`) over these values:

Hyperparameter	Values Searched
<code>max_features</code>	{2, 3, 10, 20}
<code>max_depth</code>	{2, 4, 8, 16, 32}
<code>min_samples_leaf</code>	{1, 5, 10}
<code>n_estimators</code>	{100, 500, 1000}

Table 1: Hyperparameter grid used for Random Forest parameter selection.

Brief Explanation of each hyperparameter:

- The parameter `max_features` controls the maximum number of features that the model searches over when determining the best possible split at each node.
- The parameter `max_depth` controls the maximum number of levels in each individual tree.
- The parameter `min_samples_leaf` controls the minimum number of samples a leaf in the model is allowed to contain.
- The parameter `n_estimators` controls the number of individual trees in the model.

Additionally, we set the criteria in the model to be '**entropy**', so that the trees try to minimize the weighted average entropy of child nodes when splitting. We additionally decided to implement the `class_weight` parameter which assigns more importance to classes with less samples during training. This was important as the training dataset contained 1089 samples with a requires treatment label and only 89 samples with a non treatment label. We did this using the 'balanced' function which uses this formula to create class weights:

$$\text{class_j_weight} = \text{number_samples} / (\text{number_classes} * \text{number_samples_j})$$

This means that the training samples with lesions that do not require treatment are weighted much higher when training to account for having less samples.

After performing our hyperparameter search, this was the best performing hyperparameter combination:

Hyperparameter	Best Value
<code>max_features</code>	2
<code>max_depth</code>	16
<code>min_samples_leaf</code>	1
<code>n_estimators</code>	500

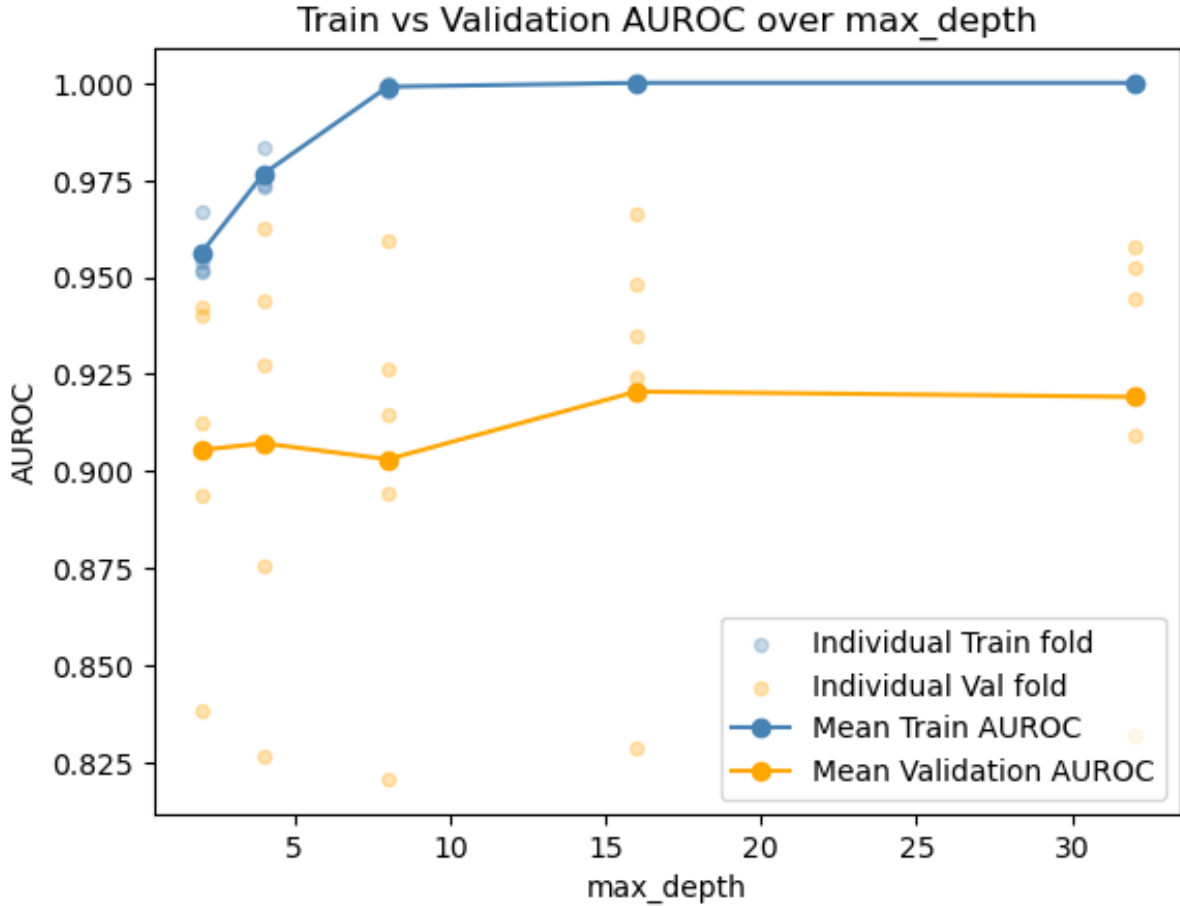


Figure 2: Hyperparameter Search Across max_depth values

The above graph shows the training and validation error of our model when varying the `max_depth` parameter, holding other parameters constant at their best value. It shows that with a low `max_depth` value such as 2 or 4, the model is underfitting and performs worse when predicting on the validation data, and as the `max_depth` value increases past 16, the model performance on the validation set begins to plateau and get slightly worse again. This signifies that with a low value the model is underfitting, and with too high a value, the model is overfitting. The graph shows that the average validation AUROC is best when the `max_depth` value is set to 16. However, from the individual validation fold data, there is some uncertainty about whether or not a `max_features` value of 8 would possibly perform better depending on which folds best represent completely unseen data. An interesting part of this graph is that above a value of 8, any model performs perfectly on the training data. This may signify that individual trees in our model could be making too specific splits, allowing the model to perfectly split the training data.

For this problem, one of the advantages of the random forest model is that it can easily handle features that contain both numerical, and binary data. Additionally, a random forest should not overfit to the data when adding more trees, as long as individual trees are not overfitting to the data. We choose our range of hyperparameters as we believed it gave the model a lot of options for the forest to either contain individual trees with high depth that may split the data into leaf nodes with small number of samples, as well as shallow individual trees that do not make as many splits. In addition, we intentionally searched over a wide number of estimators in the forest as we

were not concerned about the number of estimators contributing to overfitting.

1.4 Reflecting on Your Test Set Performance Our AUROC score on the test set is 0.90436.

Compared to the average AUROC on our validation set during cross validation, this was a very close number. Unlike in project 1, this seems to indicate that our model is doing a good job at generalizing to the overall dataset. However, there are definitely places in our model which we can improve. For example, right now, our model only factors in the age as a individual feature, but does not implement anything to treat certain age groups separately. This means that it is possible our model is performing much worse on certain age groups.

After analyzing the feature importance, age was the second most important feature when it came to tree splits in our model. This further signifies that it might be a good idea to implement strategies to maximize AUROC on age subgroups.

Problem 2: Open Ended Challenge

2.1 Feature Representation Description For this model, we utilized the image and the tabular features mentioned in Problem 1 before concatenating them with additional features created using a CNN.

1. To extract deeper visual features beyond raw OpenCV implementations, we utilized a pre-trained **MobileNetV2** model (3.5 million parameters). We utilized the model as a fixed feature extractor by removing the terminal fully-connected layers of the convolutional Neural Network and passed the images through a Global pooling layer to reduce the final convolutional volume to a 1280-dimensional embedding.
2. To prevent the new columns generated by the CNN from overfitting, we applied Principal Component Analysis to compress the image features down to 50 components, adding them with the fully numerical tabular features to create the final input feature representation.

Additionally, from the scikitlearn library we implemented:

1. **KNNImputer** was added to fill in missing values by leveraging K-nearest neighbors. We chose to do this instead of the default missing implementation as it preserves local feature relationships by estimating missing values based on similar samples.
2. **StandardScaler** was used to prevent features with larger magnitudes from disproportionately dominating the variance calculation done in PCA.

2.2 Cross Validation Description We utilized the same approach (Figure 1) for Problem 2 except with a slight modification with the **Pipeline** implementation:

1. To prevent data leakage, all preprocessing steps were placed inside the **Pipeline**, guaranteeing that the imputer, scaler, and PCA only learned distributions for the current training fold during each cross-validation split.

Additionally, instead of using the typical **roc-auc** as our performance metric for optimization, we decided to optimize over the objective that we are scored on, that being maximizing the minimum AUROC of a particular age group. Our intuition was that our model could be misleading if it performs particularly well on an age group or miserably on another. By enforcing the minimum AUROC to be high, we make our model choose hyperparameters that generalize over all age groups.

2.3 Classifier Description with Hyperparameter Search We transitioned from a Random Forest to an XGBoost ensemble architecture to better handle the non-linear relationships in the multimodal feature space. Hyperparameters were optimized using **GridSearchCV** across the following search space:

Learning Rate: The learning rate (shrinkage) controls the magnitude of the update contribution from each successive tree. We evaluated a wide range to determine if a low learning rate (favoring a robust ensemble of weak learners) or a higher rate (prioritizing faster convergence) yielded a better trade-off between bias and variance.

N-estimators: This parameter defines the total number of boosting rounds. Since each tree is trained to correct the residual errors of the previous ensemble, increasing the number of estimators improves the model’s ability to fit the training data, though it requires regularization to prevent overfitting.

Max Depth: This dictates the complexity of individual base learners. While a greater depth allows the model to capture more intricate interactions between clinical and visual features, lower depths are generally preferred to ensure the model generalizes effectively to unseen test samples.

Hyperparameter	Values Searched
learning_rate	{0.01, 0.05, 0.1}
n_estimators	{100, 500, 1000}
max_depth	{4, 6, 8}

Table 2: Hyperparameter grid for XGBoost optimization.

Similarly with Problem 1, we iterated through the available hyperparameters using `GridSearchCV`. For this model, we used the `scale_pos_weight` parameter to weight the samples to deal with class imbalance. We set the weight value too:

$$\text{scale_pos_weight} = \frac{\text{number of negative samples}}{\text{number of positive samples}}$$

This means that the training samples with lesions that do not require treatment are weighted much higher when training to account for having less samples.

After performing our hyperparameter search, this was the best performing hyperparameter combination:

Hyperparameter	Best Value
Learning Rate	0.05
max_depth	4
n_estimators	1000

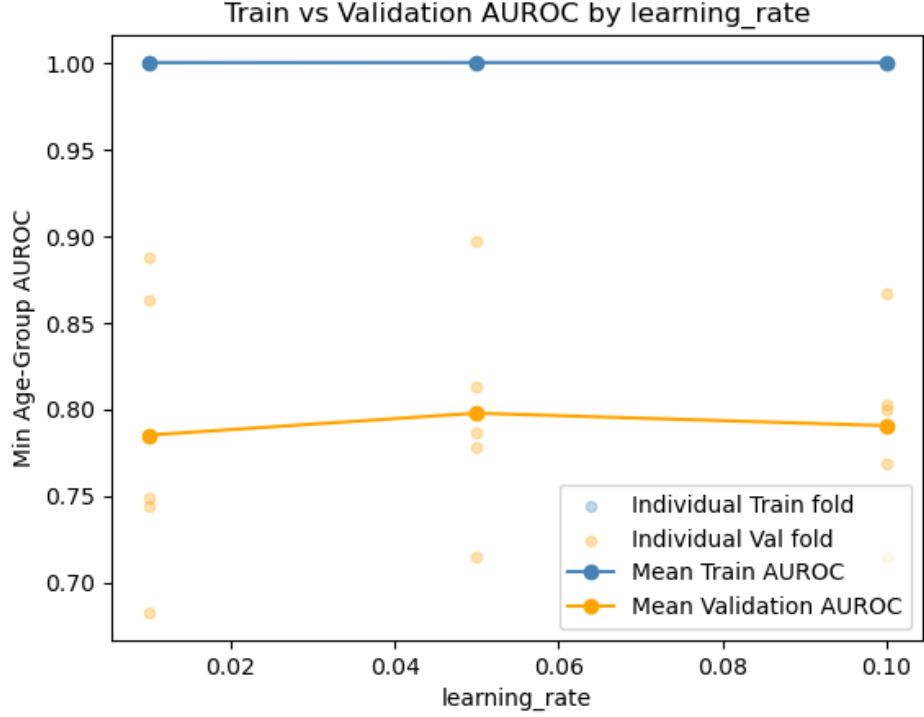


Figure 3: Hyperparameter Search Across learning rate values

We see that the "sweet spot" for generalization occurs for learning rate being 0.05. At this value, the model achieves the highest validation auroc score. From a geometric perspective, the step size was large enough to escape local minima but small enough to converge in a higher-performing region of the loss surface. We see when the grid search iterated over the 0.1 eta value, validation performance began to degrade. The increased spread of the individual validation folds at this level suggests that the optimization process is transitioning to a more volatile state.

We can also depict a clear limitation on just analyzing the learning rate hyperparameter iteration: The high training AUROC. Perhaps for future improvements on closing the validation-training performance gap, we increase regularization parameters, decrease max-depth value etc, to prevent individual trees from overfitting. Additionally, the graph does imply that the learning rate value may not be the most important hyperparameter, as the individual fold values are relatively similar.

2.4 Reflecting on Model Performance Our model in part one had an overall AUROC score of 0.90436. Our model in part two has an overall AUROC score of 0.90476, and a minimum AUROC of 0.84659 on the 60 plus year old subgroup.

Model 1 most confident correct and incorrect predictions:

Most Confident & CORRECT

Patient ID: 1131
Age: 60 | Itch: 1.0 | Bleed: 1.0
True Label: 1 | Predicted: 1
Confidence: 0.9715



Patient ID: 206
Age: 68 | Itch: 1.0 | Bleed: 1.0
True Label: 1 | Predicted: 1
Confidence: 0.9713



Patient ID: 775
Age: 68 | Itch: 1.0 | Bleed: 1.0
True Label: 1 | Predicted: 1
Confidence: 0.9712

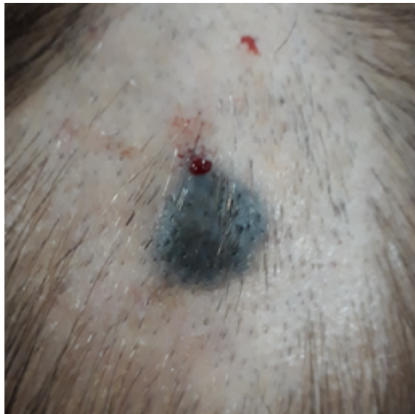


Most Confident & WRONG

Patient ID: 159
Age: 90 | Itch: 1.0 | Bleed: 0.0
True Label: 0 | Predicted: 1
Confidence: 0.8724



Patient ID: 67
Age: 66 | Itch: 1.0 | Bleed: 0.0
True Label: 0 | Predicted: 1
Confidence: 0.7977



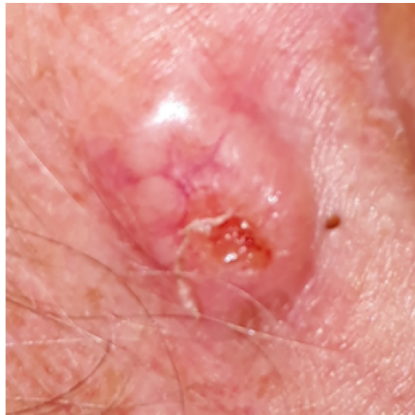
Patient ID: 463
Age: 64 | Itch: 1.0 | Bleed: 1.0
True Label: 0 | Predicted: 1
Confidence: 0.7927



Model 2 most confident correct and incorrect predictions:

Most Confident & CORRECT

Patient ID: PAT_163
Age: 63 | Itch: True | Bleed: True
True Label: 1 | Predicted: 1
Confidence: 0.9999



Patient ID: PAT_577
Age: 52 | Itch: True | Bleed: True
True Label: 1 | Predicted: 1
Confidence: 0.9999



Patient ID: PAT_101
Age: 85 | Itch: True | Bleed: True
True Label: 1 | Predicted: 1
Confidence: 0.9999



Most Confident & WRONG



Relative Performance:

1. **Problem 1:** The model accurately identified BCC and NEV cases where the color variation and circularity were very much defined. However, it identified MEL as NEV and SCC as SEK. Judging from the photo, since the dangerous skin lesion didn't exhibit extreme asymmetrical behavior, suggesting that the engineered image features were too basic in capturing other underlying features.
2. **Problem 2:** The model successfully captured the false cases and were extremely confident on them, suggesting that the CNN captured the spatial patterns effectively. However, for the malignant cases, it was extremely confident and still wrong, meaning that perhaps a feature was missing differentiating the positive/negative case

Summary: The multimodal XGBoost pipeline (Problem 2) consistently outperformed the Random Forest (Problem 1) in every metric. From an analysis standpoint, the CNN-extracted features provide a much richer representation of texture, boundary complexity, etc than the manual engineered features from OpenCV. Both models struggle with lower-contrast lesions that blend into the skin's regular pigmentation. This seems to occur more often in the older subgroup, which leads to our models poorer performance on the 60 and up age group. Perhaps the PCA compression in Problem 2 may have discarded such counter measures, and simple thresholding from Problem 1 created this issue.

Improvements: Attention-based Fusion rather than concatenating all of our features may improve performance, allowing the model to choose the weights of the tabular features or the image data to be more influential for a specific instance rather than generalizing a weight across all patients. Regularization, smaller depth, less trees could generalize more on unseen data.

AI Disclosure: We used LLMs (Claude) to help generate the code to produce our flow chart representation of the cross validation.